



Efficient Development of Multi-Agent Conversational Systems with Local and Remote LLMs and the Model Context Protocol

Eduardo Figueredo Pacheco

Institute of Mathematics, Statistics and Computer Science (IME-USP)

University of São Paulo

Supervisors: Prof. Fábio G. Cozman, Dr. Mairieli Wessel



Abstract

This work introduces a secure and modular multi-agent architecture grounded in the **Model Context Protocol (MCP)** and compatible with both local open-source LLMs and remote provider APIs. As presented in Figure 1, the system integrates three main layers: a realistic airline backend (FlightCo API), an MCP tool server exposing structured operations, and a multi-agent reasoning layer responsible for planning, tool usage, and conversation handling.

The main goal is to evaluate whether enterprise-grade agents can be reliably implemented with local LLMs while preserving safety, determinism, traceability, and operational consistency. Experiments compare latency, accuracy, and response stability across several local and cloud models, demonstrating that local LLMs are viable for secure, on-premise conversational automation.

Background and Motivation

The motivation behind this work is to move beyond LLMs used only for FAQs, simple guidance, or conversational support. The goal is to build **practical, useful AI agents** that can safely perform real actions, interacting with APIs, executing operations, and automating workflows.

This requires a secure and standardized way for agents to use external tools. Previously, such integrations were fragile and unsafe, but the **Model Context Protocol (MCP)** now provides a reliable bridge between agents and real systems. This project follows that approach by implementing a custom MCP server, giving the developer full control over what the agent can execute and ensuring a secure, enterprise-ready environment.

Research Objectives

- Develop a complete tool-based multi-agent architecture (Figure 1).
- Compare local and remote LLMs in structured tool-consumption workflows.
- Implement a reproducible environment (API + tools + agents).
- Evaluate accuracy, latency, and reliability across standardized tasks.

System Methodology

The methodology follows a bottom-up engineering approach: (1) creation of a realistic backend (FlightCo API), (2) implementation of an MCP tool server exposing structured operations, and (3) a modular multi-agent reasoning layer capable of orchestration, planning, and tool execution.

Multi-Agent Orchestration Layer

The multi-agent layer coordinates all reasoning steps. A **Planner Agent** interprets user intent and formulates the next high-level action, while the **Executor (MCP Agent)** handles low-level structured tool invocations. A lightweight **FAQ/Router Agent** manages conversational and non-actionable queries, reducing load on heavy LLMs. This decomposition improves reliability, minimizes hallucinations, and creates a predictable flow of decisions.

MCP Server Layer

The **MCP server** securely bridges LLMs and real systems. It exposes structured tools, validates inputs, blocks unsafe actions, and guarantees fully auditable, standardized interactions across models.

API / Backend Integration Layer

The **FlightCo API** simulates a realistic enterprise backend, including authentication, flight lookup and refund rules. All models interact with the same logic, ensuring fair and consistent evaluation.

Results and Discussion

LLM Performance Overview

Both local and remote models performed effectively within MCP's structured tool workflow. Remote models offer much lower latency but come with higher operational costs due to token-based pricing. Local models eliminate these costs, though they require stronger hardware to achieve competitive performance. Each option brings distinct advantages depending on budget and deployment needs.

Quantitative Summary

Latency differences are significant, yet accuracy remains consistently high across all models. Token usage remains similar because the multi-agent prompts are standardized, showing that infrastructure and cost, not correctness, drive the main trade-offs.

Table 1. Average Performance Metrics Across LLMs

LLM Model	Avg Latency (s)	Avg Total Tokens	Accuracy
Remote Models			
llama-3.1-8b-instant	0.913	2182.35	0.94
qwen/qwen3-32b	1.725	2080.99	0.75
llama-3.3-70b-versatile	1.015	2706.07	0.91
gpt-4o-mini	2.357	2185.37	0.97
gpt-4-turbo-2024-04-09	3.714	2171.04	0.99
gpt-5-mini-2025-08-07	13.176	2918.12	0.88
Local Models			
llama3.2:3b	28.721	2110.76	0.91
llama3:8b	75.142	2552.79	0.82
myaniu/qwen2.5-1m:7b	69.486	2339.32	0.88

Conclusion

MCP enables reliable tool execution in both local and remote setups. The choice depends on cost and hardware constraints, but either approach supports secure and predictable enterprise agents. Both solutions prove that tool-driven AI agents are ready for real operational use today.

System Architecture Overview

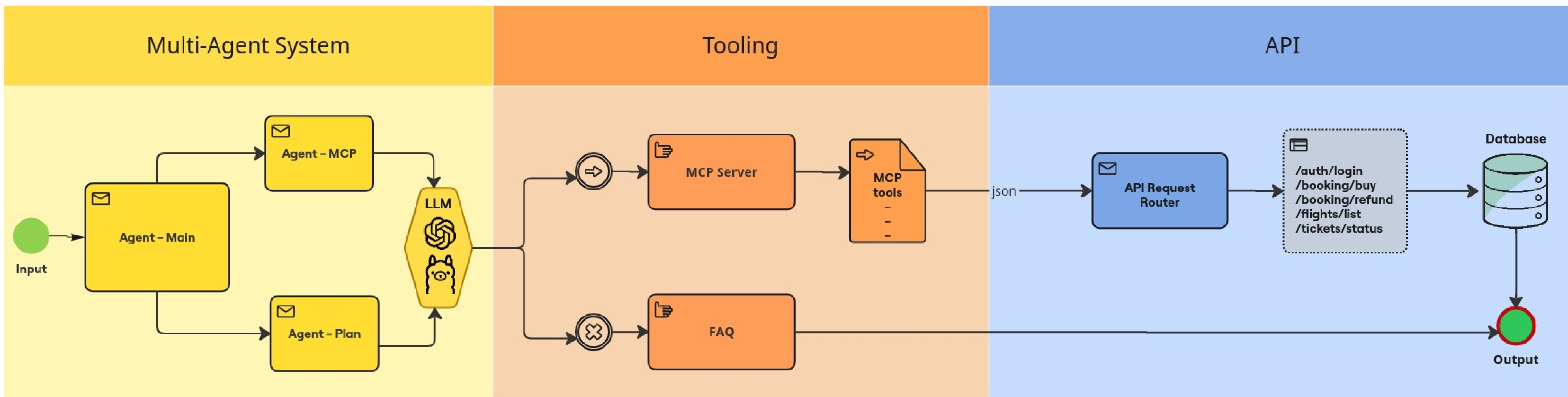


Figure 1. **System Architecture Overview:** The architecture integrates a multi-agent reasoning layer, an MCP tooling layer, and the FlightCo backend, forming a secure, reproducible, and modular environment for evaluating enterprise-ready LLM agents.